

Moon Cat (MOONCAT)

Reflection / fee-on-transfer ERC-20 · Robinhood Chain (Chain ID 4663)

Report ID	MOONCAT-SR-2026-07
Contract address	0x853587c7952a47C2E2bEEc97E2ba8d51Ca83bEfc
Source	MoonCat.sol · verified on Blockscout (robinhoodchain.blockscout.com)
Compiler	Solidity 0.6.12 · optimizer enabled, 200 runs
Contract type	Reflection / fee-on-transfer ERC-20
Total supply	1,000,000,000 MOONCAT (fixed)
Transaction fee	5% total — 1.5% reflections, 1.5% liquidity, 1.5% charity, 0.5% burn (hard-capped at 10%)
Review date	11 July 2026 (reflects the deployed, verified bytecode)
Methodology	Manual line-by-line review · fork-based functional & edge-case testing · arithmetic & reentrancy analysis · centralization enumeration

Disclaimer. This is an independent, AI-assisted security review conducted with Claude (Anthropic), following the severity taxonomy and disclosure format used by professional audit firms. It is **not** a certification by CertiK, Hacken, or any third-party audit company and must not be represented as one. A security review reduces risk; it does not eliminate it and does not guarantee a contract is free of vulnerabilities. This document is not financial advice and is not an endorsement of the project or a recommendation to transact. Always independently verify the on-chain source code.

1 · Executive Summary

MoonCat is a reflection token in the SafeMoon lineage: a fixed one-billion supply with a flat 5% transaction fee divided into holder reflections, automatic liquidity provisioning, a treasury allocation, and a burn. This review examined the complete contract for fund-loss vectors, honeypot patterns, arithmetic safety, reentrancy exposure, and privileged (centralization) powers.

No critical, high, or major-severity vulnerabilities were identified. The contract implements none of the classic honeypot mechanisms: there is no mint function, no blacklist, no transfer pause, and no facility to set a one-sided or unbounded tax. Aggregate fees are constrained to a maximum of 10% by an on-chain constant that no privileged role can exceed. The remaining residual risk is a **bounded centralization risk** — the owner may adjust fee and limit parameters within the fixed caps until ownership is renounced or transferred to a timelock (Finding MC-01, rated Minor because no fund-loss vector is present). During the review, a synchronous auto-swap that could block transfers (MC-03), an arithmetic defect in a view-only helper (MC-08), and

residual template identifiers (MC-09) were identified and **remediated prior to deployment**; the corrected contract was recompiled, re-tested (including dedicated edge-case tests for the auto-swap fix), and is the bytecode deployed and source-verified on Robinhood Chain.

0 CRITICAL	0 MAJOR	3 MINOR	3 INFORMATIONAL	3 RESOLVED
----------------------	-------------------	-------------------	---------------------------	----------------------

2 · Findings Overview

ID	TITLE	SEVERITY	STATUS
MC-01	Bounded owner control over economic parameters (centralization)	Minor	Acknowledged
MC-02	Auto-liquidity LP tokens permanently locked in contract	Minor	Acknowledged
MC-03	Failing auto-swap could block a transfer	Minor	Resolved
MC-04	setPair unrestricted and emits no event	Minor	Acknowledged
MC-05	Legacy compiler (Solidity 0.6.12)	Informational	Acknowledged
MC-06	Some administrative setters lack events	Informational	Acknowledged
MC-07	Reflection accounting gas-heavy with many exclusions	Informational	Acknowledged
MC-08	Operator-precedence bug in reflectionFromToken()	Minor	Resolved
MC-09	Leftover template identifiers in revert strings	Informational	Resolved

3 · Detailed Findings

MINOR

MC-01 · Acknowledged (Centralization Risk)

Bounded owner control over economic parameters

The `onlyOwner` account may modify the four fee components (each bounded by the 10% aggregate cap), toggle the transaction tax, adjust the maximum-transaction and maximum-wallet limits, set the charity wallet, designate addresses as tax-exempt, and reassign the tracked liquidity pair. The realistic worst case is an owner raising fees to the 10% ceiling or redirecting the charity fee.

This finding is rated **Minor** because it exposes **no fund-loss vector**: the powers that would enable theft — minting, blacklisting, pausing transfers, arbitrary balance transfers, or an unbounded tax — are **not present** in the contract, and the 10% aggregate fee ceiling is enforced by an immutable on-chain constant that no party can raise. The condition is inherent to this token category and is disclosed transparently to holders.

Recommendation: Renounce ownership (a one-transaction `renounceOwnership()` call is available and, once executed, freezes every owner power permanently and closes this finding entirely) or transfer ownership to a timelock once launch parameters are finalized. Burning the initial liquidity, which the team performs at launch, eliminates the separate liquidity-withdrawal vector.

MINOR

MC-02 · Acknowledged

Auto-liquidity LP tokens are permanently locked in the contract

Within `addLiquidity()`, LP tokens generated by the automatic liquidity mechanism are directed to `address(this)`. The contract exposes no function to withdraw them, rendering these LP tokens permanently inaccessible.

Assessment: This is not a vulnerability. Locked LP is strictly safer for holders than owner-withdrawable LP and effectively reinforces the project's burned-liquidity posture. Documented for transparency; no code change required.

RESOLVED MC-03 · Resolved

A failing auto-swap could block transfers

`swapAndLiquify()` executes inside `_transfer` when the contract's accumulated fee balance crosses the configured threshold. In the original code the call was synchronous, so a revert of the router call under an extreme liquidity condition would revert the triggering user transfer with it.

Resolution: The auto-swap was made **non-blocking** — it is now invoked via a guarded `try/catch` external self-call, so a failed swap is caught and the user's transfer completes normally. The function is additionally protected by a `require(msg.sender == address(this))` guard preventing external invocation. Verified by a dedicated edge-case suite (router forced to revert → transfer still succeeds and fee tokens are retained; external caller correctly rejected; swap resumes once the router recovers).

MINOR MC-04 · Acknowledged

`setPair` is unrestricted and emits no event

The owner may reassign `uniswapV2Pair` to an arbitrary address. While this does not move funds, the pair address governs the max-wallet exemption and the auto-swap trigger, so an erroneous change could disrupt trading. The function also emits no event.

Recommendation: Emit an event on modification and treat as an operational-safety consideration. Covered by the same renounce-ownership mitigation as MC-01.

INFORMATIONAL MC-05 · Acknowledged

Legacy compiler (Solidity 0.6.12)

The contract targets the 0.6.x series and depends on `SafeMath` for overflow protection. Solidity 0.8 and later provide built-in overflow checks and additional safety features.

Assessment: `SafeMath` is applied consistently throughout, so overflow safety is preserved. A compiler upgrade is a future consideration, not a launch blocker.

INFORMATIONAL MC-06 · Acknowledged

Some administrative setters lack events

`setTaxless`, `setPair`, and `setMinTokensBeforeSwap` alter state without emitting events, reducing off-chain observability. Fee, tax-toggle, wallet-limit, and treasury-wallet changes do emit events.

Recommendation: Add events to the remaining setters for complete transparency. Non-blocking.

INFORMATIONAL

MC-07 · Acknowledged

Reflection accounting is gas-heavy with many excluded accounts

`_getReflectionRate()` iterates over every excluded account on each transfer; a large exclusion list increases per-transfer gas cost.

Assessment: Inherent to the reflection design. The launch configuration excludes no accounts, leaving the loop empty and the cost negligible. No code change required.

RESOLVED

MC-08 · Resolved

Operator-precedence bug in `reflectionFromToken()`

The fee divisor was written `10**_feeDecimal + 2`, which under Solidity operator precedence evaluates to $(10^2) + 2 = 102$ rather than the intended $10^{(2+2)} = 10000$. The affected function is `view`-only and is not used in transfer or fee accounting — which correctly uses `10**(_feeDecimal + 2)` — so no funds were ever at risk. However, any external integration calling the helper would have received an incorrect value.

Resolution: Corrected to `10**(_feeDecimal + 2)` prior to deployment. Verified by recompilation and re-execution of the full functional test suite.

RESOLVED

MC-09 · Resolved

Leftover template identifiers in revert strings

Two `require` statements carried a foreign project name (`"Mrinity: ..."`) inherited from the forked base template, and one message was semantically inaccurate.

Resolution: Rewritten to accurate `"MoonCat: ..."` messages prior to deployment.

4 • Honeypot & Rug-Vector Assessment

The contract was evaluated against the specific mechanisms commonly used to trap buyers or extract liquidity:

VECTOR	PRESENT	NOTES
Mint / inflation function	No	Supply fixed at construction; no mint path
Blacklist / sell-blocking	No	No per-address gating on transfers
Transfer pause / trading switch	No	Tax may be toggled; transfers cannot be halted
Unbounded or one-sided tax	No	Symmetric on buy/sell; hard-capped at 10%
Arbitrary balance drain / sweep	No	No owner function transfers user funds
Hidden owner-only sell path	No	Identical code path for all sellers
Proxy / upgradeable logic	No	Immutable, non-upgradeable implementation
Owner-adjustable fees	Yes	Present but bounded $\leq 10\%$ — see MC-01

5 • Verification Performed

- Line-by-line manual review of the transfer, fee-collection, reflection-rate, and swap-and-liquify logic.
- End-to-end functional testing (18 assertions) executed against the exact production bytecode on a local fork of Robinhood Chain mainnet using the live Uniswap V2 router — covering deployment and pair creation, liquidity provision, taxed buys and sells, the precise 5% fee split, reflection accrual to a passive holder, the automatic swap-and-liquify path, and every transaction-limit and administrative access control. **All 18 assertions passed.**
- Arithmetic reviewed for overflow (SafeMath coverage) and operator precedence; reentrancy surface examined around the lock-guarded auto-swap and the payable receive function.
- Privileged (owner) capabilities enumerated and assessed against the honeypot and rug-vector checklist above.

6 • Conclusion

The Moon Cat contract is a conventional reflection token that, following remediation of the three identified issues, presents no critical, high, or major-severity security vulnerabilities and none of the recognised honeypot mechanisms. Its only remaining item is a bounded centralization risk (MC-01, Minor), which is standard for the category and is mitigated by the immutable 10% fee ceiling, the absence of mint/blacklist/pause/drain capabilities, the burning of initial liquidity at launch, and the availability of a one-transaction ownership renouncement that closes the finding entirely. Prospective participants should note that bounded owner privileges exist until ownership is relinquished, and should independently verify the deployed source code on the block explorer.

